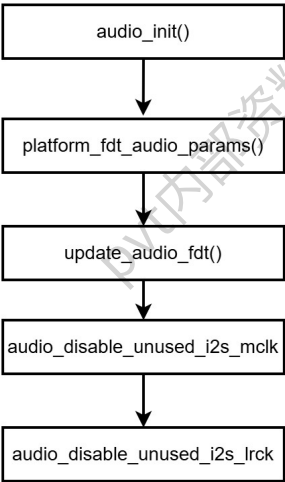


LK AUDIO 配置

1.1 目录架构

目录	描述
yocto/src/bsp/lk2/platform/mediatek/mt8676/audio	lk audio 初始化代码

1.2 代码流程



1. audio_init() :初始化 I2S1/4/6 MCK、BCK, 参数来自于结构体 struct mtk_afe_i2s_priv

```
struct mtk_afe_i2s_priv {
    int id;
    int rate; /* for determine which appl to use */
    const char *clk_ao_property_name;
    const char *mclk_ao_property_name_lk;
    const char *lrck_ao_property_name_lk;
    int share_i2s_id;
    int mclk_id;
    int mclk_rate;
    int mclk_appl;
    int mclk_pdn_offset;
    enum CLK_AO mclk_ao;
    enum CLK_AO lrck_ao;
    int ch_num;
    int sync;
    int ip_mode;
    int bit_len;
    int enabled;
};
```

2.platform_fdt_audio_params() : 读取 dts compatible = "mediatek,mt6897-sound" 节点，获取 clk-always-on 参数，并对 i2s_priv.mclk_ao、i2s_priv.lrck_ao 标志位赋值，该标志位表示是否开启 clk always on 。

```
i2s_priv[i2s_id].mclk_ao = mclk_ao;
i2s_priv[i2s_id].lrck_ao = lrck_ao;
i2s_priv[i2s_id].mclk_rate = mclk_ratio * lrck;
if (i2s_priv[i2s_id].mclk_rate > 24576000)
    i2s_priv[i2s_id].mclk_rate = 24576000;
i2s_priv[i2s_id].rate = lrck;
i2s_priv[i2s_id].bit_len = lrck_width;
i2s_priv[i2s_id].ch_num = channels;
if (share_i2s_id >= 0) {
    i2s_priv[share_i2s_id].mclk_ao = mclk_ao;
    i2s_priv[share_i2s_id].lrck_ao = lrck_ao;
    i2s_priv[share_i2s_id].mclk_rate = i2s_priv[i2s_id].mclk_rate;
    i2s_priv[share_i2s_id].rate = lrck;
    i2s_priv[share_i2s_id].bit_len = lrck_width;
    i2s_priv[share_i2s_id].ch_num = channels;
}
```

3. update_audio_fdt(): 判断上面函数读取到的 DTS 信息，并更新 DTS 节点信息，供驱动识别，防止在

kernel 节点出现多次开启 clk 的情况。

4. audio_disable_unused_i2s_mclk()/audio_disable_unused_i2s_lrck(): 通过 DTS 信息判断是否打开多余的 CLK，有则关闭。

```
void audio_disable_unused_i2s_lrck(void)
{
    audio_disable_unused_i2s();
    audio_disable_unused_i2s_lrck_gpio();
    audio_disable_unused_i2s_bck_gpio();
}

void audio_disable_unused_i2s_mclk(void)
{
    audio_disable_unused_i2s_mclk_cg();
    audio_disable_unused_i2s_mclk_gpio();
}
```

1.3 客制化步骤

以下新增一个 I2S2 作为 demo，理解以上的流程。

1. 先使能 I2S2 的 CLK、GPIO 等信息，由于 8676 I2SOUT 都是共用的 I2SIN CLK，所以需要对齐 I2SIN2、I2SOUT2 配置：

CONFIG: 需对齐 I2SIN2、I2SOUT2 的 rate、ch_num、ip_mode、share_i2s_id 等参数。ip_mode 表示 data line，0 表示 one data line multi-channel，1 表示 multi data line 2-channel。share_i2s_id 表示需要使用哪个 i2s 的 clk，目前是使用 I2SIN2 的 CLK。具体的 CONFIG 说明详见寄存器文档：《3.12 Audio System.pdf》。

I2S MODE: 可配置为 I2S 格式或 TDM 格式，目前仅 I2S4 可配置为 TDM 格式，寄存器值为 0x4/0x5。

GPIO: 对照硬件设计，确认对应的 GPIO NUM 和 I2S MODE，此前遇到的一个 I2S CLK 延后输出，就是因为此处的 I2S MODE 配置错误，导致未按预期输出 CLK。



2. DTS 配置

按照如下格式，在 DTS 添加相关属性，该属性作用如下：

```
--- a/recipes-kernel/linux/files/auto8676p1_64_hyp.dts
+++ b/recipes-kernel/linux/files/auto8676p1_64_hyp.dts
@@ -850,6 +850,7 @@
    etdmout6-clk-always-on = <0xf 0x2ee00 0x80 0x20 0x2>;/* ao_level, lrck, mclk_ratio, bclk_width, channels */
    etdmout1-clk-always-on = <0xf 0xbb80 0x200 0x20 0x2>;/* ao_level, lrck, mclk_ratio, bclk_width, channels */
    fmi2s-clk-always-on = <0xf 0xbb80 0x02 0x20 0x2>;/*ao_level, lrck, mclk_ratio, bclk_width, channels */
+    etdmout2-clk-always-on = <0xf 0xbb80 0x200 0x20 0x2>;/* ao_level, lrck, mclk_ratio, bclk_width, channels */
};
```

(1). lk 识别后, 不会关闭该 I2S CLK, 如上述的 audio_disable_unused_i2s_lrck 函数。

(2). kernel 识别到该属性后, 不会重复开关 I2S CLK。

```
int mt6897_apll1_enable(struct mtk_base_afe *afe)
{
    struct mt6897_afe_private *afe_priv = afe->platform_priv;
    int ret;

    #if IS_ENABLED(CONFIG_LK_I2S_AO_CLK_SUPPORT)
    if (is_i2s_ao_clock_enabled_apll(afe, MT6897_APLL1) && afe_priv->i2s_force_setting == 0) {
        dev_info(afe->dev, "%s() lk enabled i2s, already enabled apll1\n", __func__);
        return 0;
    }
    #endif
}
```

3. kernel 开关

可在驱动中 打开路由开关, 该数组会在驱动加载完成后设置为 Dummy_Widget, 并避免设置为 Normal 导致 CLK 被关闭的情况。

```
--- a/sound/soc/mediatek/mt6897/mt6897-mt6338.c
+++ b/sound/soc/mediatek/mt6897/mt6897-mt6338.c
@@ -45,6 +45,7 @@ static struct {
 } clk_always_on kctl_table[] = {
     {"PLAYBACK_DUMMY_TO_I2SOUT_Mux"},
     {"I2S_OUT4_Mux"},
+    {"I2S_OUT2_Mux"},
     #if IS_ENABLED(CONFIG_FM_I2S_CLK_AO_SUPPORT)
     {"FM_I2S_IN_Mux"},
 }
```

也可以在 android audio_device.xml 中打开, 如下:

```
index d919e45..9268a16 100755
--- a/noboauto8676_platform/audio_device.xml
+++ b/noboauto8676_platform/audio_device.xml
@@ -1456,6 +1456,7 @@
     <kctl name="I2S_OUT4_Mux" value="Dummy_Widget" />
     <kctl name="I2S_OUT1_Mux" value="Dummy_Widget" />
     <kctl name="I2S_OUT6_Mux" value="Dummy_Widget" />
+    <kctl name="I2S_OUT2_Mux" value="Dummy_Widget" />
     </path>
     <path name="MD1_TO_CAPTURE2" value="turnon">
```

4. configure 配置

目录:

yocto/meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/auto8676p1_64_hyp_defconfig

```
CONFIG_GRT_HYPERVISOR_SVP=m
CONFIG_MAGIC_SYSRQ_SERIAL=n
CONFIG_MTK_DRAM_BOOST=y
CONFIG_LK_I2S_AO_CLK_SUPPORT=y
CONFIG_MTK_ADSP_AUTO_MULTI_AUDIO_HAL_SUPPORT=y
CONFIG_ANDROID_FIX_PCIE_SLAVE_ERROR=y
CONFIG_XGBE=m
```